TAL Project Report

Milos Oundjian Vladislav Antipin

Year: 2025-2026

Contents

1 Introduction 3
2 Datasets 3
  2.1 Movie Reviews 3
  2.2 President Speeches 4
3 Methods 5
  3.1 Preprocessing 5
    3.1.1 Movie Reviews 5
    3.1.2 President Speeches 5
  3.2 Feature Representations 8
    3.2.1 Bag of Words / TF-IDF 8
    3.2.2 Word Embeddings 9
  3.3 Classical Models 10
    3.3.1 SVM 10
    3.3.2 Logistic Regression 11
    3.3.3 Ensemble Models 11
  3.4 Neural Models 11
    3.4.1 Recurrent Neural Networks 11
    3.4.2 Transformers 11
  3.5 Post-Processing 11
  3.6 Training and Evaluation 12
    3.6.1 Evaluation Strategy 12
    3.6.2 Metrics 12
    3.6.3 Tuning of Hyperparameters 13
4 Results and Discussion 14
  4.1 Movie Reviews 14
  4.2 President Speeches 17
5 Conclusion 19
6 Appendix 20

1 Introduction

The goal of this project is to implement and evaluate different natural language processing methods in the context of two binary classification tasks:

1. Movie review sentiment analysis: predict whether a given review shows a positive or negative sentiment.
2. President speech authorship attribution: predict whether a sentence belongs to Jacques Chirac's or François Mitterrand's speech.

These tasks are representative of two common types of linguistic signals - emotional and stylistic, as well as of preprocessing and evaluation challenges summarized in Table 1.

Characteristic	**Movies**	**Presidents**
Signal	Emotional	Stylistic
Language	English	French
Speech	Informal	Formal
Classification Unit	Paragraph (Review)	Sentence
Observations	Assumed i.i.d.	Sequential Dependence
Class Balance	Balanced	Imbalanced

Table 1: Task comparison.

2 Datasets

Composition	**Movies**	**Presidents**
0 label	1000 (negative)	49890 (Chirac)
1 label	1000 (positive)	7523 (Mitterrand)
Labeled dataset size	2000	57413
Unlabeled dataset size	25000	27162
Mean number of characters (labeled dataset)	3861	134
Mean number of words (labeled dataset)	668	23

Table 2: Dataset descriptions.

2.1 Movie Reviews

The labeled dataset consists of 2000 samples of movies reviews split evenly between positive and negative reviews - 1000 of each (Table 2).

This labeled dataset comes from the Cornell Movie Review Data dataset. The reviews in this dataset have already been slightly preprocessed, having been down-cased and cleaned of HTML tags (Table 3).

Training set	plot : two teen couples go to a church party , drink and then drive .
Test set	Yes its an art... to successfully make a slow paced thriller.

Table 3: Example sentences from movie review training and test sets. Each review is made up of multiple sentences. Labeled set has spaces before punctuation and unlabeled set has leftover HTML tags.

The positive/negative rating labels in this dataset were also automatically implemented based on star reviews with the system they used explained in the dataset readme. This could lead to some mistakes in labels.

The unlabeled dataset is much longer and contains 25,000 reviews. It is less preprocessed than the labeled dataset retaining capital letters and still containing some HTML tags. On average the reviews in the unlabeled dataset are shorter (1325 vs the 3860 of the labeled dataset).

2.2 President Speeches

The training and test sets represent corpora of political speeches of J. Chirac with segments of speeches of F. Mitterrand, taken mainly from the site of Elysée.

Each document represents a sentence with average length of 23 words (Table 2). Order of documents in both train and test sets preserve the actual order of sentences in speeches, however they don't constitute a coherent discussion between presidents but rather a mix of speeches. This temporal dependency was exploited in postprocessing of predicted probabilities.

The major challenge of this dataset is that it is highly unbalanced - Mitterrand documents are almost seven times less frequent. This issue was addressed by adjustment of class weights and rescaling of predicted probabilities.

As can be seen for examples from Table 4, documents for both training and test sets come preprocessed, with dates and names being replaced by special tokens.

Training set	<107:42:M> Nous ne pouvons ni attendre une embellie incertaine, ni nous résigner. <107:43:C> <nom> , vous avez bien servi les armées de la France.
Test set	<105:2> Et ce moment exceptionnel, j'ai essayé de le concrétiser dans un geste qui soit un peu hors du commun : le retour à Alger du sceau du <nom> . <105:3> Et le musée.

Table 4: Example documents from presidents training and test sets.

3 Methods

3.1 Preprocessing

3.1.1 Movie Reviews

In the case of the movies dataset, the labeled dataset is already somewhat preprocessed as explained earlier. In order to have the models trained on the labeled dataset generalize better to the unlabeled data we choose to normalize the text in both, opting for making all words lowercase, and cleaning up HTML tags in the unlabeled dataset. This was implemented through regex and Python string manipulation functions.

Stop words are common words that carry little meaning that can be chosen to be removed from text, we choose not to always remove them in the preprocessing and instead and keep the removal of them to be a hyperparameter in our TF-IDF vectorizer.

As shown in Figure 1, most common words include mostly linked to movies in general ("movie", "film", "story"), as well as value judgement words such as "good" and "like". Words that appear more often in positive or in negative reviews are mostly movie and actor names.

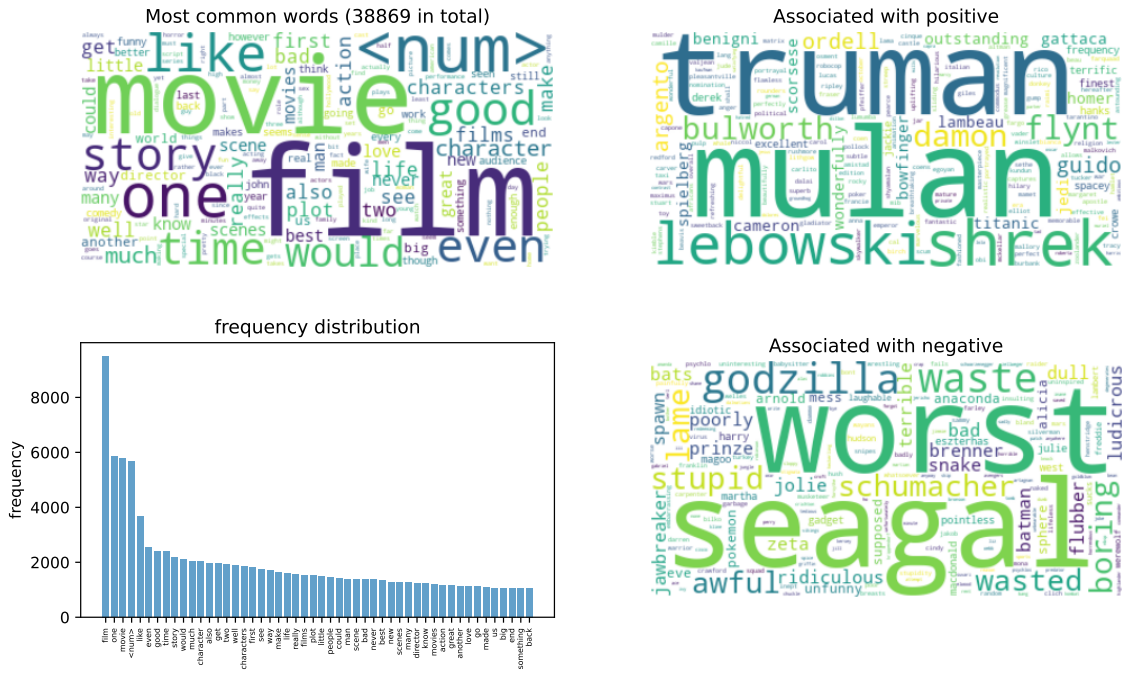


Figure 1: Most common and most discriminative words for movie reviews. Wordcloud and histogram on the left show the distribution of most common words. Wordclouds on the right show words with high (top right) and low (bottom right) odds ratios. Odds ratios are computed as ratio of odds for a given word to be found in positively rather then negatively labelled document. Stop words were excluded.

3.1.2 President Speeches

The task of authorship attribution relies on mainly stylistical habits and less on semantic content, which makes this task different from sentiment analysis. Although the presidents

have political preferences, they are speaking about semantically close topics, mainly French and international politics.

It can be thus of interest to include character n-grams, keeping punctuation and capitalization. However, given that those texts are transcriptions of oral speeches, including this information can identify president's transcribers rather than themselves. We decided to try both approaches and compare the results. Of note, the most common transformer for classification of French texts, CamemBERT, is cased by default and thus can implicitly exploit discrepancies in uppercase usage in those transcriptions.

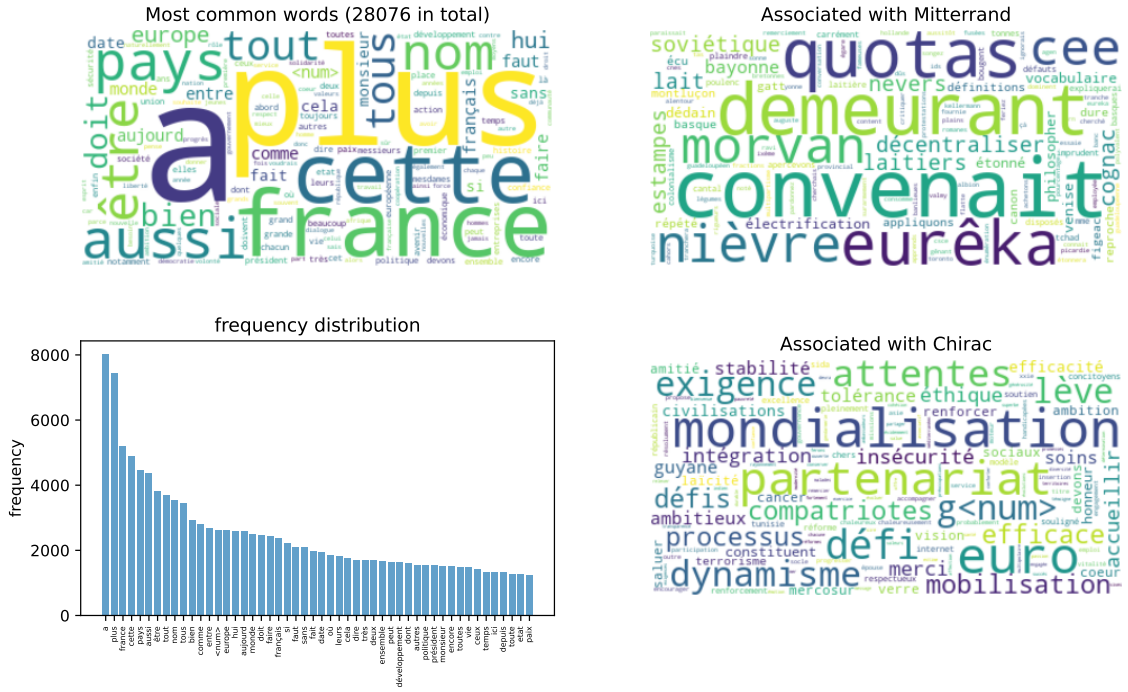


Figure 2: Most common and most discriminative words for presidents speeches. Same conventions as for Figure 1.

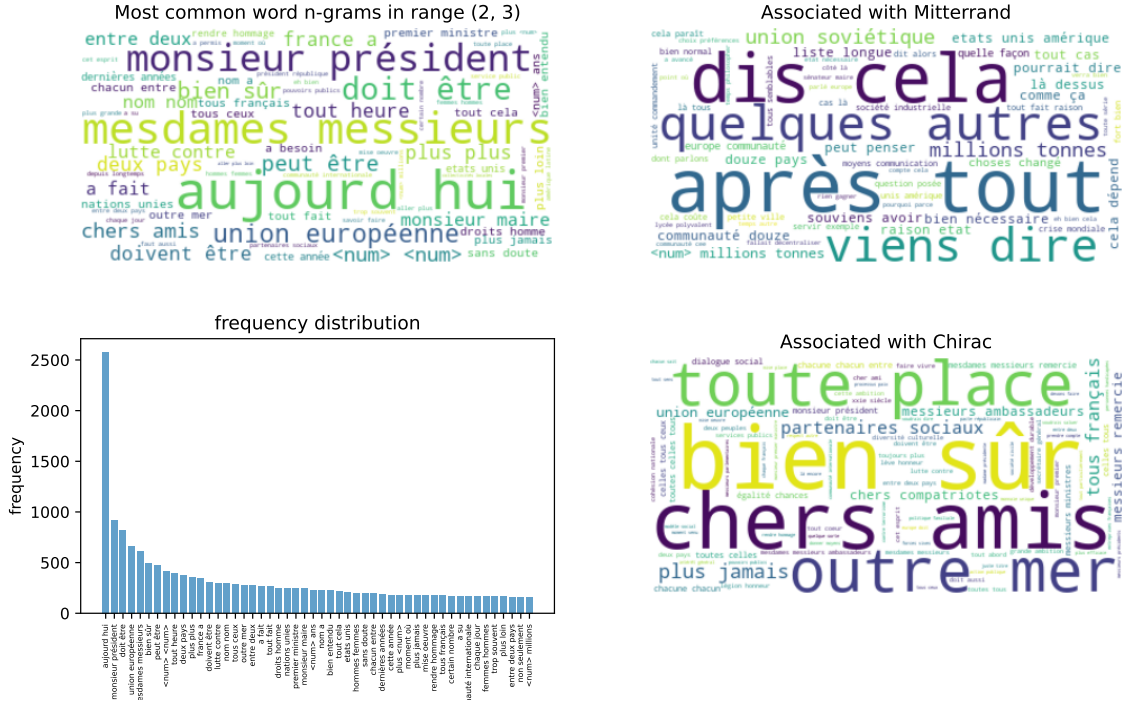


Figure 3: Most common and most discriminative word bi- and tri- grams for presidents speeches. Same conventions as for Figure 1.

As can be seen in Figure 2, after the removal of stop-words the Presidents dataset has the vocabulary size of 28,076 words, the most frequent of them are still some common French words such as “plus” or “aussi”, and words related to the politics, such as “France”, “pays”.

Figure 2 shows as well words with highest odds ratios for Mitterrand and for Chirac. The corresponding words may approximately reflect preferred terms and topics for those politicians. For example, word “quotas” being associated with left-wing Mitterrand, and “mondialisation” with right-wing Chirac.

Among common word n-grams (Figure 3) can be found mostly curtesy phrases and greetings (e.g. “mesdames messieurs”, “monsieur president”, “chers amis”), linking phrases and discourse markers (“après tout”, “bien sûr”), as well as composite names of political institutions (“union europeene”, “union sovietique”, “états unis amérique”). Most common bi-gram is actually a single composite word “aujourd’hui” which could be added as an exception to pre-processing. As we can see, the word bi-gram level has a potential to represent stylistical habits of presidents and thus should be considered for further analysis.

Character-level n-grams are also interesting, since they can show patterns in punctuation usage, or preference towards specific word forms. But we must be careful with those features since they can be related mostly to the way the speech was transcribed and thus are less interesting.

Experiments with stemming didn’t have a significant effect on downstream analysis and are not shown.

3.2 Feature Representations

3.2.1 Bag of Words / TF-IDF

Bag of words approaches work by eliminating the sentence structure by turning each document into a vector in the vocabulary space. Their main advantage is that due to their simplicity, they are very fast both in training and in inference. This allows us to do cross-validation. However, such representation is sparse since each document contains only a small fraction of total vocabulary.

The simplest approach is to consider term frequences in each document, using `CountVectorizer`. However, this method gives too much importance to the words that appear consistently across documents and don't carry any discriminative signal, such as "the", "la", etc.

Term frequency–inverse document frequency (TF-IDF) solves this problem by scaling term frequences by inverse document frequencies, down-weighting common words:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \left(\log \left(\frac{1 + n}{1 + \text{df}(t)} \right) + 1 \right)$$

Although simple, TF-IDF combined with a regularized linear model is still considered as a good baseline for classification.

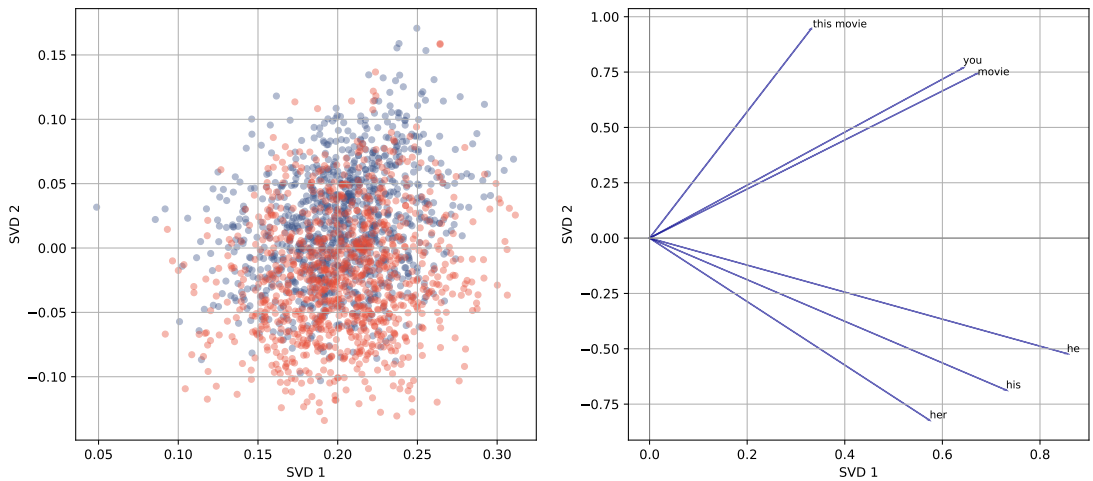


Figure 4: Latent Semantic Analysis: SVD on TF-IDF for movie reviews. On the left, sample projections onto first two SVD components. On the right, scaled loadings of features contributing the most to the first two components.

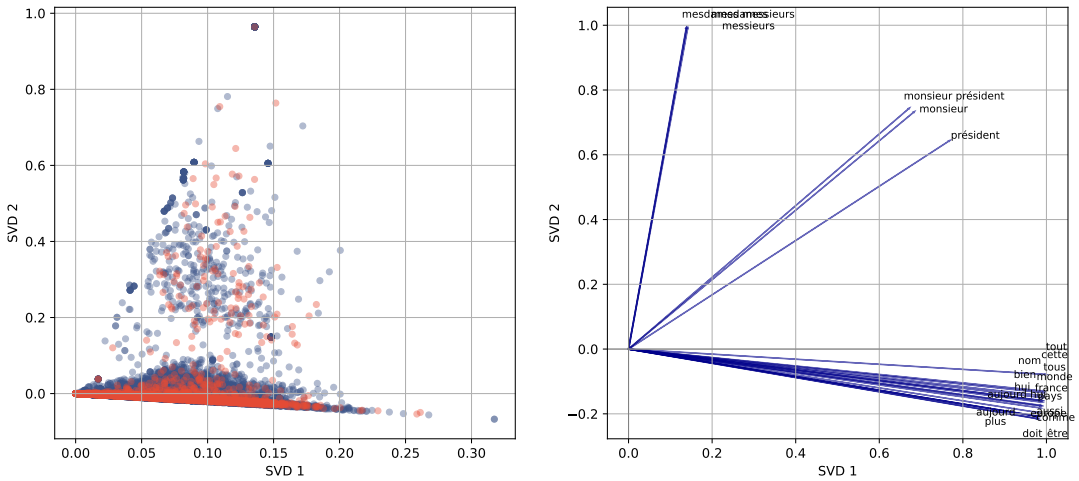


Figure 5: Latent Semantic Analysis: SVD on TF-IDF for president speeches. Same conventions as for Figure 4.

The Latent Semantic Analysis consists in the SVD performed on TF-IDF document vectors and allows us to explore the dominant co-occurrence patterns in the text corpus.

Figure 4 shows us that the major "topics" for movies dataset include third-person pronouns "he", "her", "his" (apparently referring to movie characters or actors) and the "movie" topic itself. On Figure 5 we can see that the first "topic" relates to some general discussion about "France", "monde" and "pays", whereas the second one relates to greetings such as "mesdames", "messieurs". It is coherent with the fact that the president speeches are fragmented at sentence level and greetings are usually reserved for first and final sentences. We can as well see that for both datasets these topics are not discriminatory of the target label, which means that the classification signal is not directly linked to the general corpus structure.

3.2.2 Word Embeddings

Bag of words representations suffer from sparsity, high-dimensionality and lack of semantic information captured. Embeddings, on the contrary, are dense word / document representations in a lower (300-900) dimensional space that are obtained in a way to capture semantics. One way to obtain them is by predicting the context from a word (Skip-gram) or vice-versa (CBOW), which is exploited by `Word2Vec`. `GloVe` obtains embeddings by factorization of a co-occurrence matrix obtained from big corpora of texts.

We used `fastText`, an extension of `Word2Vec`, handling subword structure and thus is more flexible, especially for French language. We can visualize the most discriminatory words, obtained by computing odds ratios, in the latent space of fastText embeddings with dimension reduction technique UMAP to see whether they cluster semantically. Figure 6 shows us that most discriminatory words form clusters of actors (e.g. Ordell, Prinze in the lower left cluster), and less distinct regions for movies (e.g. Shrek, Mulan, Godzilla), as well as regions linked to the positive sentiment ("memorable", "masterpiece", "fantastic") and negative sentiment ("worst", "terrible", "boring"). Overall, we can see that sentiment is indeed associated with discrimination between movie reviews, however, sentiment can be attributed directly to movies and actors (e.g. people tend to like Shrek and dislike Godzilla).

On Figure 6 we see less clustering and semantic regionalization of discriminatory words, except for a small region related to “défi”, “mobilisation” and “dynamisme” characteristic of left-wing Mitterrand. Overall, we can see that discrimination between presidents would require a richer context-dependent semantic and stylistic representations.

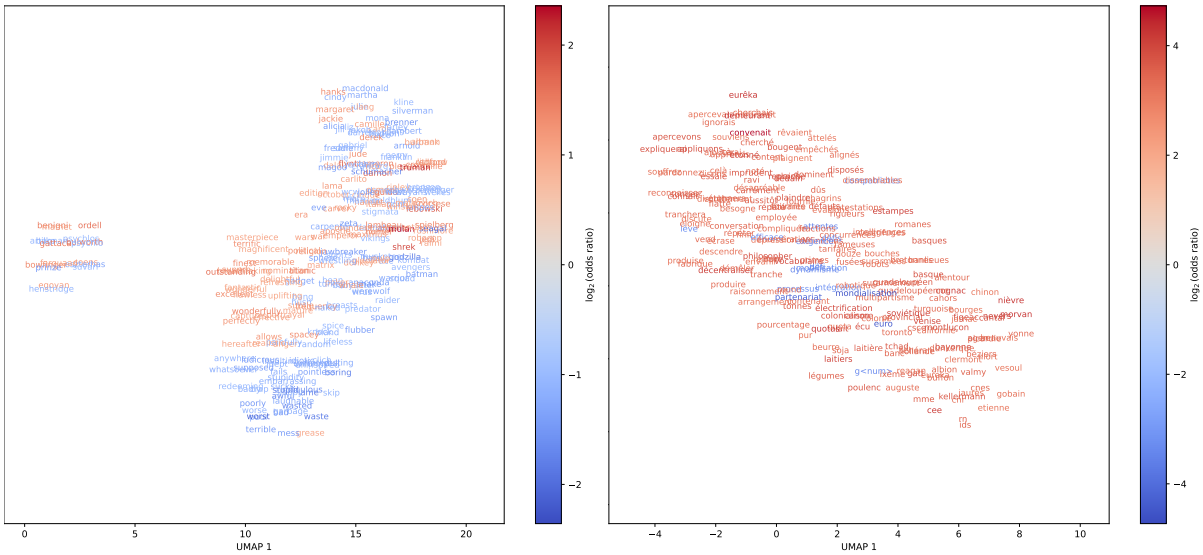


Figure 6: UMAP on fastText embeddings of most discriminative words from movie reviews (on the left) and from president speeches (on the right). For visual clarity, only the features with log2 of odds ratio above 1 (for movies) and 3 (for presidents) were projected.

However, document classification requires document embeddings. They can be obtained by mean, min and max aggregation of corresponding word embeddings, but this method is not robust to long documents and doesn’t capture contextual information. This is why we considered as well embeddings of <CLS> tokens obtained by forward pass through the attention layers of the RoBERTa and CamemBERT transformer models.

3.3 Classical Models

The choice of classifier depends on the type of vector representations used. High-dimensional sparse representations such as TF-IDF usually result in a $p > n$ problem, and thus the usage of complex non-linear models is discouraged due to over-fitting. We expect simple linear regularized models perform the best on BoW data. Dense representations are less dimensional, and we could therefore find more complex non-linear patterns with non-linear models such as XGBoost.

3.3.1 SVM

LinearSVC is a type of Support Vector Machine that finds a linear boundary to separate classes. It works well for text data because TF-IDF represents documents as high-dimensional vectors, which linear models handle efficiently. In Scikit-learn, we can combine TfidfVectorizer and LinearSVC in a Pipeline, so the text is automatically converted to TF-IDF features and then classified in one step.

The specific advantage of LinearSVC in Scikit-learn is that it is very fast to fit, however it comes with the downside of not being able to use non-linear kernel functions. Although kernel functions are not as important in text classification problems since the dimensions of text-vectors are very high and as such linear boundaries are usually expressive enough.

Another implementation of SVM, SVC in Scikit-learn, allows for probability outputs by fitting a sigmoid to the SVM decision scores (Platt scaling). However, with `probability=True`, the model is slower to fit since it requires internal cross-validation.

3.3.2 Logistic Regression

Logistic Regression is a similar linear classifier that fits a linear decision boundary, but instead of maximizing the margin like SVMs, it models the conditional probability of each class using a sigmoid function. This means it can directly output class probabilities, which can be used for ROC curves and threshold adjustments. Similarly to LinearSVC, it works quite well with TF-IDF features and can be used in a Scikit-learn Pipeline with TfidfVectorizer.

3.3.3 Ensemble Models

Ensemble models such as Random Forest or XGBoost are expressive enough to fit complex non-linear decision boundaries and by construction are more resilient to overfitting. They are expected to perform the best on dense vector representations.

3.4 Neural Models

3.4.1 Recurrent Neural Networks

Recurrent neural networks are based on a learned sequential transformation of inputs into hidden states and then task-specific outputs. Back propagation through time results in vanishing gradient, which degrades the performance for long texts. This problem is addressed in models such as LSTM and GRU with additional mechanisms for memory tracking. We thus have decided to consider only the latter.

3.4.2 Transformers

Transformers are based on a iterative contextual enrichment of token embeddings and special tokens (e.g. `<CLS>` for BERT) through a mechanism of self-attention. They are expected to perform better, especially for sentiment analysis.Since we have a classification task, we used BERT architecture, specifically RoBERTa for movie reviews and CamemBERT for president speeches tasks.

We also decided for experimental purposes to try zero-shot learning with a generative transformer, specifically Qwen3.5:9b, a 9 billion parameter open source model.

3.5 Post-Processing

The presidents dataset represents a sequence of sentences from speeches and thus the index of each document carries useful information. The sentences that are next to each other have a high chance of belonging to the same author, we can use this prior information to smoothen the predictions by a convolution with the Gaussian window (optimal window size was found to be between 17 and 21).

Another challenge with presidents dataset is that it is unbalanced - positive class is almost 7 times less present than negative. Among tested approaches, the best one was the combination of balancing class weights for the loss function and adjustment of predicted probabilities to shift the prediction threshold towards the majority class, thus favoring the minority class.

Under-sampling the majority class and over-sampling the minority class with `imblearn` package had a slight improvement to the performance, but not as good as threshold adjustment.

For the convenience of Scikit-learn pipeline fitting and cross-validation, we implemented a `SmoothEstimator` class with modified `predict()` and `predict_proba()` methods that allow temporal smoothing and threshold adjustment.

3.6 Training and Evaluation

3.6.1 Evaluation Strategy

Cross-validation is a model assessment technique used to estimate a model's generalization performance. The goal of cross validation is to be able to use the full data for training and testing. Instead of using on a single train-test split, the dataset is partitioned into several folds, and the model is trained and evaluated multiple times, each time using a different fold as validation and the remaining folds for training. This reduces the variance of the evaluation and provides a more reliable estimate of model performance.

The inconvenience of this is that it requires fitting the model once for each fold, so for example 5-fold cross validation will take five times as long as a single train-test split. In our case this means that we will only use cross-validation when the models are fast enough to train.

When doing model hyperparameter tuning and model selection, another layer of cross-validation is better so as to not overfit the hyperparameters to the specific folds on which we are evaluating our model. This nested-cross validation is used for the hyperparameter tuning of the TF-IDF models on the movies dataset.

Since presidents task requires the intact order of samples for the postprocessing, train-validation split and K-fold cross-validation splits should not be random. Thus, for presidents we used 5-fold cross validation without shuffling and for train-validation split we have taken 20% of last examples in the dataset for validation. The obvious downside of that approach is that there is no stratification possible and the validation set might be not representative enough of the whole dataset. Indeed, the validation set has a slightly higher imbalance (minority class is almost 9 times less represented), and therefore the performance estimates will be probably slightly lower than on a real test set. However, given that a major performance gain comes from temporal smoothing and threshold adjustment, these parameters need to be able to be cross-validated as well.

3.6.2 Metrics

For a classification task, the main evaluation metrics are accuracy, precision, recall, and F1-score. Accuracy measures the overall proportion of correct predictions, while precision focuses on how many predicted positives are actually correct, and recall measures how many true positives are successfully identified. The F1-score combines precision and recall into a single

value by taking their harmonic mean, aiming to provide a single measure that takes both into account.

In the case of the movies reviews dataset, thanks to it being balanced, these four metrics are usually quite close, meaning that using accuracy as our "main" metric is a possible choice.

However for the president speeches dataset, the uneven class distribution makes it so that accuracy can be a misleading metric, as such it is better to consider metrics such as F1-score, area under the ROC curve (ROC-AUC) and average precision (AP, area under Precision-Recall curve). Among those metrics, only F1-score depends on the threshold choice.

3.6.3 Tuning of Hyperparameters

Hyperparameter tuning consists of changing the hyperparameters (parameters of the model that are fixed before the model is fitted) in order to get better performance. There are several different ways of hyperparameter tuning such as grid search and random search.

Since hyperparameter tuning requires fitting or training the model multiple times, it is more feasible on simpler models and smaller datasets. In the case of the movies dataset, these are the TF-IDF based models.

For the movies dataset the hyperparameters we chose to tune are shown in Table 5. We are limited in the number of values tune over as the time cost is $O(2^n)$ in the number of values we choose to tune over.

Hyperparameter	Values	Best Value (Both Models)
N-gram Range	`[(1, 1), (1, 2)]`	`(1, 2)`
Min df	`[1, 3]`	`3`
Max df	`[0.9, 1.0]`	`0.9`
Sublinear TF	`[True, False]`	`True`
Stop Words	`[None, "english"]`	`None`
C (Regularization)	`[1, 10]`	`10`

Table 5: Hyperparameter grid for TF-IDF + LinearSVC and TF-IDF + Logistic Regression on movie reviews.

We found that for both models the best parameters were the same with the results shown in Table 5.

Given that the presidents training set is larger, the hyperparameter tuning with cross-validation becomes slow when it comes to simultaneous tuning of multiple parameters, we thus used `RandomSearchCV` with 100 iterations over a reasonable range of parameters, listed in Table 6.

Hyperparameter	Values	Best Value
N-gram Range	`[(1, 1), (1, 2), (1,3)]`	`(1, 2)`
Min df	`[1, 3]`	`1`
Max df	`[0.9, 1.0]`	`1.0`
Sublinear TF	`[True, False]`	`True`
C (Regularization)	`[0.1, 1, 5, 10]`	`5`
Gaussian window size	`[15, 17, 19, 21, 23]`	`21`
Prediction threshold	`[0.35,0.4, 0.45, 0.5,]`	`0.4`

Table 6: Hyperparameter grid for TF-IDF + Logistic Regression on president speeches.

Other models (such as XGBoost, Naive Bayes) were less sensitive to hyperparameter choice and thus default hyperparameters were kept, except for those involved in class balancing.

For CamemBERT, no hyperparameter tuning was performed since this model takes the longest to train (from 30 minutes to hours). Instead, we set the size of attention window (`max_length`) to be 110 tokens, batch size for training of 32 samples and learning rate of 2.10^{-5}. The window size for smoothing is taken the same as was found for logistic regression, and extremely low prediction threshold of 0.05 based on the analysis of distribution of predicted probabilities (due to overconfidence of the model).

4 Results and Discussion

4.1 Movie Reviews

The movies dataset was the simpler of the two for getting good performances. Simply using a TF-IDF with a LinearSVC with default parameters gave 85% accuracy and ~0.85 in the other 3 metrics on 5-fold stratified cross validation. TF-IDF with a Logistic regression achieved slightly worse performance with around ~0.82 in all 4 metrics (Table 7).

The speed of fitting and predicting with these models is in the range of a few seconds for the 2000 samples in the labeled dataset and the 25000 samples in the unlabeled dataset are shown Figure 9. We find that the models each are quite fast taking only a few seconds. The tuned models are about twice as slow as the default hyperparameter models due to their use of bigrams requiring more time to vectorize data.

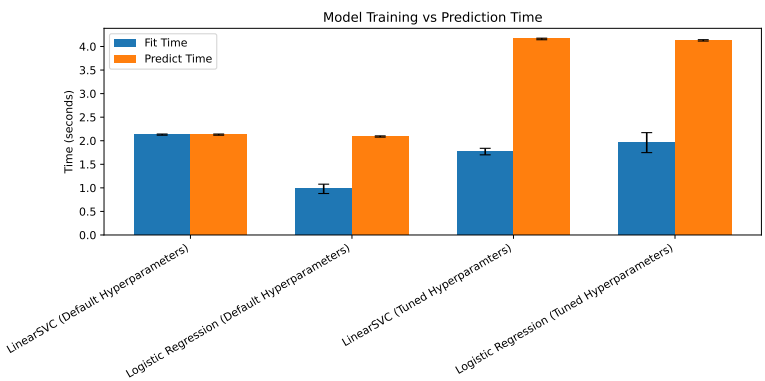
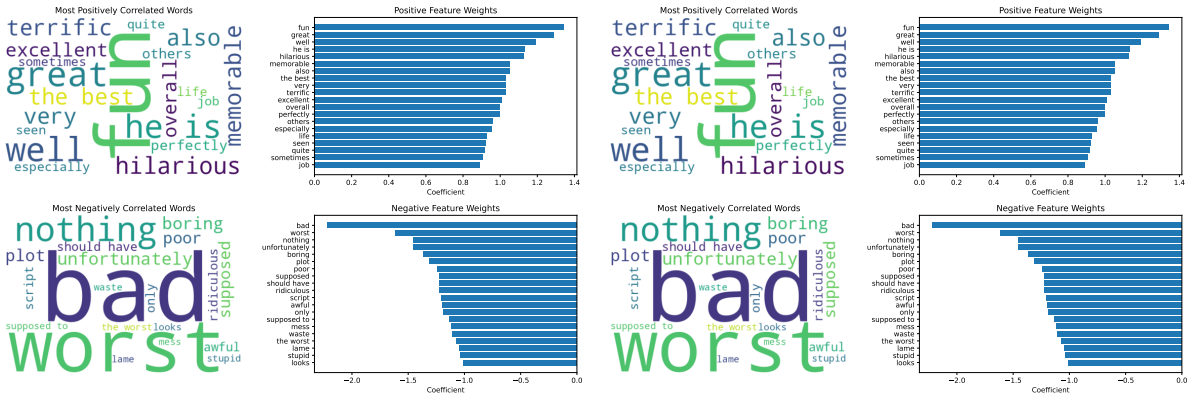


Figure 9: Time to fit models on full labeled dataset (2000 samples) and time to predict on full unlabeled dataset (25000 samples) for different TF-IDF methods.

The accuracy of these models improved to close to 90%, with all metrics around ~0.89 with tuning of the hyperparameters.

Another advantage of these methods are the interpretability. Figure 10 shows the words most correlated with the positive and negative categories. We see that the results for both LinearSVC and Logistic Regression are quite similar.



TF-IDF + LinearSVC (Tuned Parameters) TF-IDF + Logistic Regression (Tuned Parameters)

Figure 10: Wordcloud and bar chart of most positively and negatively correlated words and bigrams for tuned TF-IDF + LinearSVC and TF-IDF + Logistic Regression. We can see that both models have very similar results.

As the current state of the art models, transformers also did very well in this task achieving over 0.9 in all aspects. The ability of these models to capture contextual meaning of words allows them to understand nuances and concepts like sarcasm and long-range dependencies that simpler BoW type algorithms lose in the vectorization of the sentences.

The extensive pretraining that models like RoBERTa have give them an "understanding" of the workings of language that can be adapted to our classifcation task through fine-tuning.

The disadvantage with this model is that it is much slower than the TF-IDF models, especially for training. For prediction, even with good hardware that the transformer model can take advantage of (in our case a NVIDIA GeForce RTX 3080 on the PPTI), it still took several minutes to predict on the full unlabeled dataset.

Also based on the transformer architecture, the generative transformer Qwen model we used managed similarly high performance without any fine-tuning or looking at any examples in our dataset. However this did come at a much greater cost taking about 3 seconds to classify each review running locally on an Macbook Pro with M4 Pro chip, making it hundreds of times slower than the TF-IDF models and also slower than the RoBERTa model.

An exhaustive list of the results of the training are shown in Table 7 and Figure 15 below.

Model	Accuracy	F1-Score	Precision	Recall
TF-IDF + LinearSVC (Default Hyperparameters)	0.85	0.85	0.84	0.87
TF-IDF + LinearSVC (Tuned Hyperparameters)	0.89	0.89	0.88	0.90
TF-IDF + Logistic Regression (Default Hyperparameters)	0.82	0.82	0.82	0.83
TF-IDF + Logistic Regression (Tuned Hyperparameters)	0.88	0.89	0.88	0.89
Fine-tuned RoBERTa	0.95	0.95	0.92	0.97
Qwen3.5:9b Zero-shot	0.95	0.95	0.96	0.94

Table 7: Validation performances for models used on movie reviews. For the TF-IDF this is average performance on 5-fold stratified cross validation. For RoBERTa and Qwen3.5 this is over 400 samples which form the validation dataset for the fine-tuning of RoBERTa.

Model	Post-processing	F1	ROC-AUC	AP
TF-IDF + SVC + Platt scaling	before	0.48	0.84	0.49
	after	0.54	0.96	0.83
TF-IDF + Logistic Regression	before	0.45	0.85	0.52
	after	0.73	0.97	0.84
TF-IDF + Random Forest	before	0.16	0.80	0.32
	after	0.62	0.94	0.73
TF-IDF + XGBoost	before	0.33	0.74	0.34
	after	0.58	0.91	0.66
TF-IDF + MultinomialNB	before	0.39	0.81	0.39
	after	0.63	0.93	0.71
fastText + Logistic Regression	before	0.36	0.80	0.37
	after	0.62	0.95	0.69
fastText + XGBoost	before	0.42	0.80	0.4
	after	0.67	0.95	0.72
CamemBERT + Logistic Regression	before	0.45	0.88	0.55
	after	0.65	0.96	0.76
CamemBERT + XGBoost	before	0.48	0.85	0.5
	after	0.70	0.96	0.75
LSTM at word level	before	0.41	0.78	0.39
	after	0.51	0.94	0.75
Fine-tuned CamemBERT	before	0.61	0.91	0.68
	after	0.82	0.99	0.89

Table 8: Performances for models used on president speeches on the same validation set - contiguous subset of the last 20% of the full training set. Colors highlight remarkable results.

4.2 President Speeches

The summary of performances on presidents dataset is shown in Table 8. We notice first that the post-processing (temporal smoothing with threshold adjustment) results in a major performance gain of at least 10 percentage points across almost all metrics and all models.

As with the movies dataset, the best performing model on TF-IDF turned out to be the simplest one - ridge logistic regression with 0.84 AP after post-processing, and more expressive XGBoost performed poorer on this data, yielding in only 0.66 AP. This is due to high dimensionality and sparsity of TF-IDF representations that more expressive models tend to overfit. Naive Bayes and Random Forest have shown intermediary results with AP slightly above 0.7.

The advantage of those baseline classifiers is the interpretability of the learned weights. As shown in Figure 11, the features associated with Mitterand (with highest positive weights) and Chirac (negative weights) are related to their habits of usage of discourse markers such as "après tout", "puisque", "tout cas", "bien sûr", but as well terms related to the discussed topics as "euro", "unis amérique", or even "000" - a marker of more frequent usage of numbers. Of note, at first our model included as well character-level n-grams and captured differences in casing such as "Monsieur" vs "monsieur", finally their contribution turned out to be marginal and character n-grams were not considered in this report.

Figure 11: Words and word bi-grams associated with largest positive (on the left) and negative (on the right) coefficients for TF-IDF + LogisticRegresion.

Since we are dealing with rather short sentences, the well-known issue of "memory span" of RNN could be less prominent, we then tried using LSTM at word level, and trained it for 10 epochs. However, it gave only moderate results with 0.75 AP.

We tried as well training those baseline classifiers on dense representations. For this, we used a concatenation of `mean`, `min` and `max` aggregations of word embeddings to obtain document embeddings. Both Logistic Regression and XGBoost have shown moderate performance with AP around 0.7, although we expected XGBoost to yield better results given that dense representations allow more reliable fitting of complex decision boundaries. However, it was not the case, apparently because the aggregation of semantic information from fastText embeddings is not precise enough to discriminate presidents.

We thus decided to try the embeddings of `<CLS>` token obtained by a forward pass of a pretrained CamemBERT transformer. The overall performance is better than with fastText

embeddings with AP around 0.75, however still similar between Logistic Regression and XGBoost and moderate compared to best results on TF-IDF.

The latter result suggests that simply training the MLP head of CamemBERT classifier might not be enough to get good performance. Therefore, we decided to fine-tune last 2 transformer blocks and to train the final MLP (that is initialized randomly). To tackle the class imbalance issue, we used class weights as a square root of inverse class frequency to not punish the model too much for minority class mispredictions. Training for 10 epochs was enough to have results that outperform all other tried models, yielding AP of 0.89.

To further examine the difference between two best performing models (TF-IDF with Logistic Regression and fine-tuned CamemBERT), we plotted the distributions as well as the results of temporal smoothing for predicted probabilities (Figure 12, Figure 13 and Figure 14). We can see that Logistic Regression probabilities are rather smoothly distributed between one and zero, meaning that the model is rarely confident and gives noisy estimates. This noise is smoothened out with temporal smoothing which leads to dramatic increase in performance. However, CamemBERT predicted probability distribution is very peaked towards 0 and 1 - the model is very confident. However, when it's wrong, the wrongly predicted probabilities will be still smoothened out, but would require much stronger threshold adjustment (to around 0.05), since rare peaks at probability 1 get smoothed by a lot of neighbors that are almost exactly 0.

Although CamemBERT performed the best, it took the most time to train (around 30 minutes on NVIDIA GeForce RTX 3080 GPU) and is less interpretable.

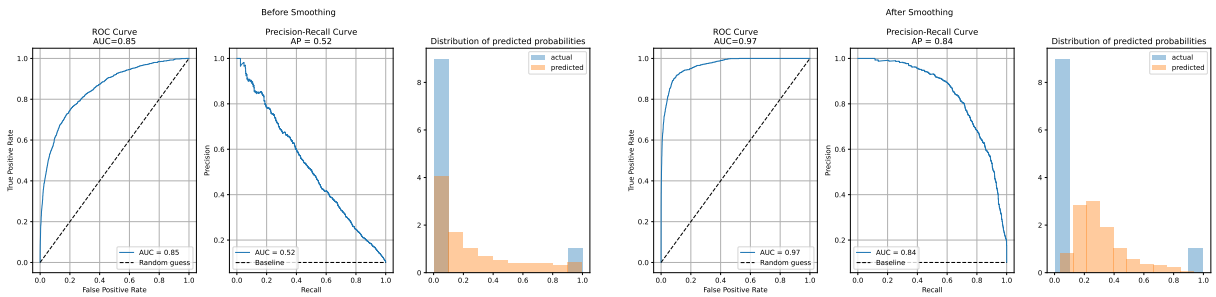


Figure 12: ROC curves, Precision-Recall curves and distribution of predicted probabilities for TF-IDF + Logistic Regression on presidents speeches before and after smoothing.

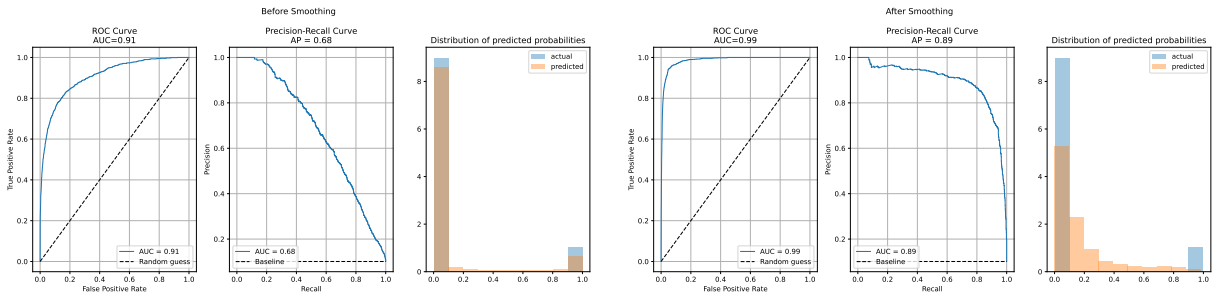


Figure 13: ROC curves, Precision-Recall curves and distribution of predicted probabilities for fine-tuned CamemBERT on presidents speeches before and after smoothing.

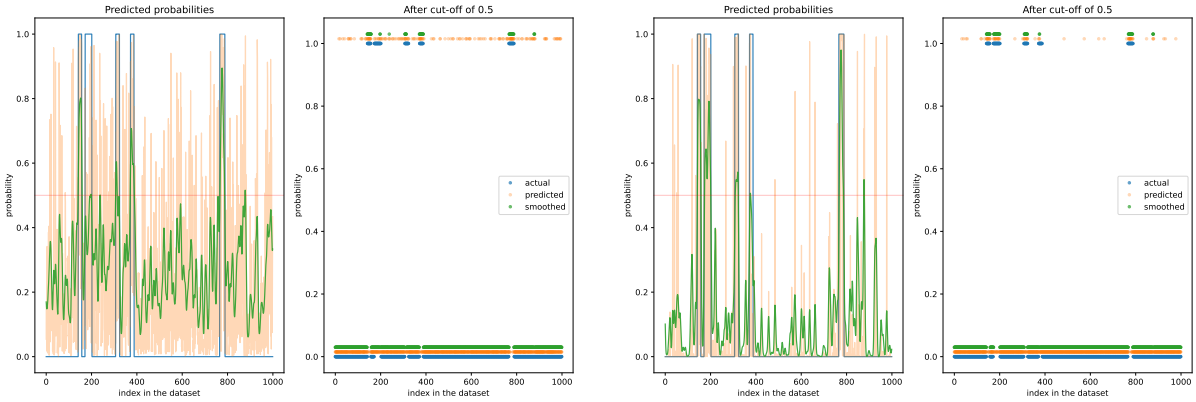


Figure 14: Fragment of true, raw and smoothed probabilities predicted by TF-IDF + Logistic Regression (on the left) and fine-tuned CamemBERT (on the right) over first 1000 examples in presidents speeches dataset.

5 Conclusion

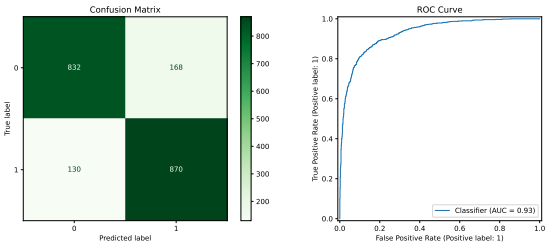
There are many factors that come into play when choosing the model that is appropriate to the task, however it seems to come down to one key trade off: Speed vs Performance.

BoW methods such as TF-IDF combined with regularized linear classifiers have very decent performances at fractions of the computing power of more complex models. Particularly in classification tasks, their performances may be more than adequate for many situations. However, in cases where mis-predictions are critical (e.g. in production, in medical field), state of the art models such as BERT can be further fine-tuned to obtain more robust results. Intermediate strategy consisting in learning baseline classifiers on pretrained document embeddings seems to be the least efficient for the tasks where the semantic or stylistic precision is needed, unless the embeddings are directly aligned with the task.

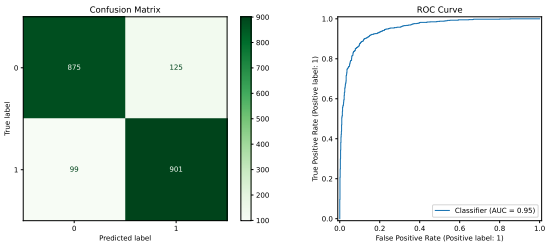
Dealing with unbalanced datasets is trickier, among tried options, setting class weights for the loss function combined with threshold adjustment seemed to be the most convenient and efficient, however, depending on the task, one can consider under- or over- sampling or even data augmentation, such as reverse translation. The latter was not tried as for the president speeches task data augmentation risks erasing fine stylistical differences between the presidents.

To conclude, we identify other approaches that it would be interesting to explore. For example, trying hierarchical RNN (first on word level and then on sentence level), HMM and CRF to model transitions of authors between sentences. Finally, the analysis of attention maps of BERT models could provide more understanding into model's decision-making process.

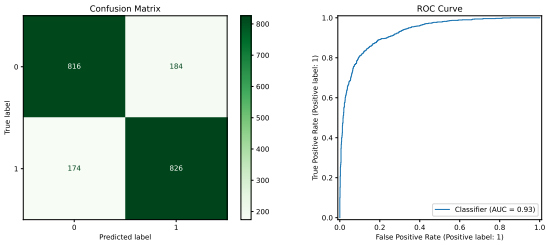
6 Appendix



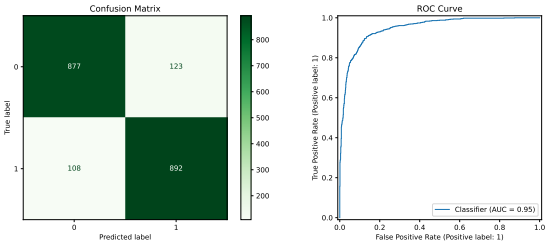
TF-IDF + LinearSVC (Default Parameters)



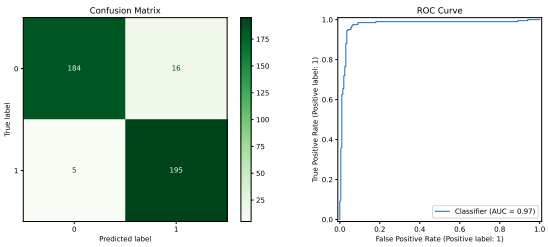
TF-IDF + LinearSVC (Tuned Parameters)



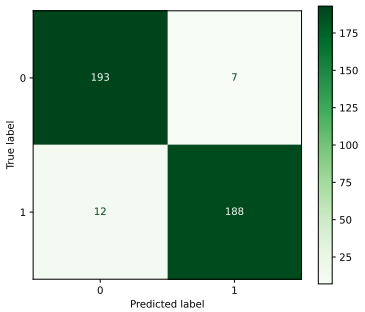
TF-IDF + Logistic Regression (Default Parameters)



TF-IDF + Logistic Regression (Tuned Parameters)



RoBERTa Fine-tuned



GPT (Qwen3.5:9b) Zero-shot

Figure 15: Confusion matrices and ROC curves for models tried on movie reviews. Since LinearSVC does not give probabilities, we use the decision score which measures the review is from the separating hyperplane for the ROC curve. Note that there is no ROC curve for the GPT due to it outputting only text in the form of “negative” or “positive”.